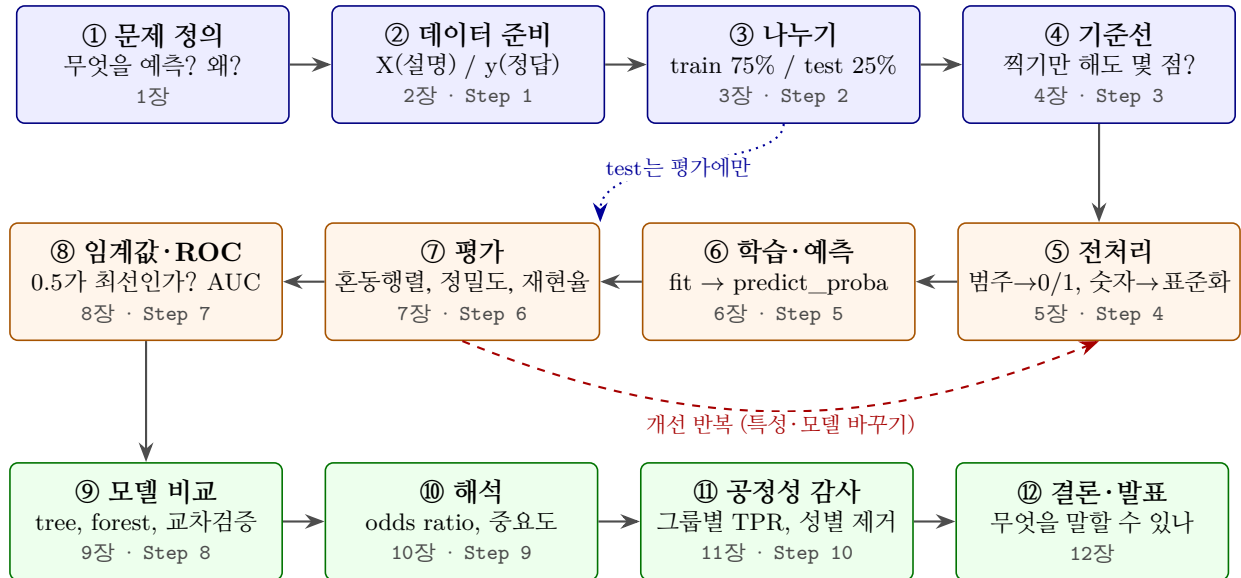


코드로 연습하는 분류 (Classification)

채용 데이터로 “합격 여부”를 예측하며 배우는 머신러닝의 전 과정

연습 노트북: `classification_practice.ipynb` (빈칸) 정답: `classification_solution.ipynb`



- 준비 (①-④) 데이터를 모델이 먹을 수 있게, 그리고 공정하게 평가할 수 있게
- 모델링 (⑤-⑧) 학습하고, 예측하고, 얼마나 맞는지 쟀다
- 검증·해석 (⑨-⑫) 다른 모델과 비교하고, 이유와 공정성을 따진다

이 워크북의 핵심 메시지

채용 데이터로 합격 예측 모델을 만들면 모델은 과거 채용의 편향까지 배웁니다. 그래서 이 워크북의 목표는 “좋은 채용 AI 만들기”가 아니라, 분류 모델을 만들어 보면서 그 모델이 어떤 편향을 재생산하는지 감사(audit)하는 것입니다. 이것이 datathon 주제 “Exploring Bias in Hiring”과 연결되는 지점입니다.

목차 (Contents)

0	이 워크북 쓰는 법	3
1	분류란 무엇인가: 문제 정의	4
1.1	분류 (classification)	4
1.2	이번 문제	4
1.3	먼저 생각할 것: 이 모델을 왜 만드나	4
2	Step 1. 데이터 준비: X와 y	5
3	Step 2. train / test 나누기	6
3.1	왜 나누나: 과적합 (overfitting)	6
4	Step 3. 기준선 (baseline)	7
5	Step 4. 전처리 파이프라인	8
5.1	두 가지 변환	8
6	Step 5. 학습과 예측	9
6.1	모델	9
7	Step 6. 평가: 혼동행렬과 지표	10
7.1	혼동행렬 (confusion matrix)	10
7.2	지표	10
8	Step 7. 임계값과 ROC 곡선	11
9	Step 8. 다른 모델과 교차검증	12
9.1	교차검증 (cross-validation)	12
10	Step 9. 해석: 계수와 중요도	13
11	Step 10. 공정성 감사 (fairness audit)	14
11.1	그룹별로 혼동행렬 쪼개기	14
11.2	성별을 빼면 해결될까? (fairness through unawareness)	14
11.3	반사실 테스트: 성별만 바꾸면?	15
12	정리와 발표	16
12.1	이번 실습으로 알게 된 것	16
12.2	발표용 문장 (예시)	16
12.3	다음에 해 볼 것	16
13	확인 질문 답과 빈칸 정답	17
13.1	확인 질문	17

13.2 빈칸 정답	17
----------------------	----

0 이 워크북 쓰는 법

준비물

- 폴더 ~/DataPythonPrep: hiring_practice.csv, classification_practice.ipynb (빈칸), classification_solution.ipynb (정답, 실행 결과 포함).
- VS Code나 Jupyter에서 classification_practice.ipynb를 엽니다.

```
cd ~/DataPythonPrep
jupyter notebook classification_practice.ipynb
```

또는 VS Code에서 열기

각 장의 구성

상자	하는 일
목표	이 단계에서 할 일 한 줄, 다이어그램의 위치
본문	필요한 개념만 짧게
코드 (빈칸)	노트북과 같은 코드. <code>___</code> 를 채워서 실행
실행 결과	실제로 돌린 출력. 내 결과와 비교
무슨 일이 일어났나	코드가 데이터를 어떻게 바꿨는지 단계별로
확인 질문	이해했는지 점검 (답은 13장)

※ 빈칸 정답은 13장 끝 “빈칸 정답”과 정답 노트북에 있습니다. 막히면 5분 고민 후 보세요.

※ 다른 책과의 연결: [흐름 n] = pandas-stats-walkthrough, [응용 n] = hiring-statistics-book, [알고 n] = algorithms-for-datathon.

회귀 분석(다른 책)과 무엇이 다른가

	통계 회귀 (응용 책)	분류 머신러닝 (이 책)
목적	설명: 성별 효과가 몇 %p인가, 유의한가	예측: 새 지원자가 합격할까
데이터 사용	전부로 추정	train으로 학습, test로 평가
성공 기준	p-value, 신뢰구간	정확도, 정밀도, 재현율, AUC
도구	statsmodels	scikit-learn

같은 logistic regression이 두 책에 다 나오지만, 묻는 질문이 다릅니다.

1 분류란 무엇인가: 문제 정의

목표 ①

“무엇을, 무엇으로, 왜 예측하는가”를 한 문장으로 정한다.

1.1 분류 (classification)

정답이 범주인 예측 문제입니다. 정답이 숫자면 **regression**(회귀)입니다.

예	정답 y	종류
이 지원자가 합격할까	0 / 1	이진 분류 (binary)
이 메일은 스팸일까	0 / 1	이진 분류
어느 부서에 맞을까	Engineering / Marketing / ...	다중 분류 (multiclass)
시험 점수는 몇 점일까	29-100 숫자	회귀

1.2 이번 문제

- 정답 y : hired (1 = 합격, 0 = 불합격)
- 설명변수 X : gender, department, degree, test_score, years_experience
- 학습이란: X 와 y 의 짝 수백 개를 보고, X 만 주면 y 를 맞히는 규칙 f 를 찾는 것: $\hat{y} = f(X)$.

1.3 먼저 생각할 것: 이 모델을 왜 만드나

- 과거 채용 결과(y)가 편향되어 있으면, 그것을 잘 맞히는 모델은 편향을 잘 배운 모델입니다. 정확도가 높을수록 공정하다는 뜻이 아닙니다.
- 그래서 이 문제의 올바른 사용법은 (1) 어떤 특성이 결과를 좌우하는지 이해하고, (2) 모델이 그룹별로 다르게 틀리는지 감사하는 것입니다 (11장).

※ 실제 사례: 2018년 한 대형 기업이 이력서 평가 모델을 폐기했습니다. 과거 채용 데이터가 남성 위주라서 모델이 “여성”과 관련된 단어에 감점을 학습했기 때문입니다.

확인 질문

1. “지원자의 예상 연봉 맞추기”는 분류인가 회귀인가?
2. 테스트 정확도 95%인 채용 모델은 공정하다고 할 수 있는가?

2 Step 1. 데이터 준비: X와 y

목표 ②

표에서 문제지(X)와 정답지(y)를 분리한다.

```
df = pd.read_csv("hiring_practice.csv")
d = df.__( )
cat = ["gender", "department", "degree"]
num = ["test_score", "years_experience"]
X = d[___]
y = d[___]
print(X.shape, round(y.mean(), 3))
```

결측 행 제거
범주형 열
숫자형 열
설명변수 5개
결과변수

실행 결과

(582, 5) 0.325

무슨 일이 일어났나

1. `dropna()`: `test_score`가 빈 18행을 지워 600 → 582행. (모델은 NaN을 받지 못합니다.)
2. `X = d[cat + num]`: 리스트 두 개를 이어 붙인 5개 열만 고른 DataFrame. `applicant_id`와 `hired`는 빠집니다.
3. `y = d["hired"]`: 열 하나 = Series.
4. `y.mean() = 0.325`: 582명 중 32.5%가 합격. 이 숫자가 4장 기준선의 출발점입니다.

※ `applicant_id`를 X에 넣으면 안 됩니다. 번호에는 의미가 없는데, 모델은 우연한 패턴을 배울 수 있습니다. [응용 2.2]

※ 정답 열(`hired`)이나 정답에서 파생된 열이 X에 섞이면 **data leakage**: 테스트 점수가 비현실적으로 높게 나옵니다.

확인 질문

3. `X = d.drop(columns=["applicant_id", "hired"])`는 위 코드와 같은 결과인가?

3 Step 2. train / test 나누기

목표 ③

모델이 본 적 없는 데이터로 평가할 수 있도록 일부를 떼어 둔다.

3.1 왜 나누나: 과적합 (overfitting)

시험 문제를 미리 보고 공부하면 그 시험은 100점이지만 실력은 모릅니다. 모델도 학습에 쓴 데이터로 평가하면 외운 것과 배운 것을 구분할 수 없습니다.

- **Training set**: 모델이 규칙을 배우는 데이터 (75%)
- **Test set**: 마지막에 한 번, 성적표를 받는 데이터 (25%). 학습·튜닝에 절대 쓰지 않습니다.

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=___, stratify=___, random_state=0)
print(len(X_train), len(X_test), round(y_train.mean(), 3), round(y_test.mean(), 3))
```

실행 결과

```
436 146 0.326 0.322
```

무슨 일이 일어났나

1. 582행의 순서를 무작위로 섞습니다 (random_state=0: 매번 같은 섞기 → 결과 재현 가능).
2. 25% = 146행을 test로, 나머지 436행을 train으로.
3. stratify=y: 합격자 비율(32.5%)이 양쪽에서 같도록 합격자/불합격자 각각 25%씩 떼어 냅니다. 그래서 0.326과 0.322로 거의 같습니다.
4. 결과는 4개: 문제지와 정답지가 각각 train/test로.

※ 통계 연결: test 점수도 146명 표본에서 나온 추정값이라 흔들립니다. 그래서 9장에서 교차검증으로 여러 번 짹니다. [흐름 4, 9]

확인 질문

4. stratify를 빼면 어떤 문제가 생길 수 있나? 특히 합격자가 5%뿐인 데이터라면?

4 Step 3. 기준선 (baseline)

목표 ④

“아무 생각 없이 찍어도” 받는 점수를 먼저 안다. 모델은 이것보다 나아야 의미가 있다.

```
dummy = DummyClassifier(strategy=__).fit(X_train, y_train)
print(round(accuracy_score(y_test, dummy.predict(X_test)), 3))
```

실행 결과

0.678

무슨 일이 일어났나

1. "most_frequent": train에서 가장 많은 답 (불합격 = 0)을 찾습니다.
2. test 146명 모두에게 0이라고 예측합니다. X는 보지도 않습니다.
3. test의 실제 불합격자 99명을 다 맞히므로 정확도 $99/146 = 0.678$.

교훈: 정확도 68%는 “아무것도 안 배운” 점수입니다. 불균형 데이터에서 정확도만 보면 속습니다. 이 모델은 합격자를 한 명도 찾지 못합니다 (재현율 0). 그래서 7장의 다른 지표가 필요합니다.

확인 질문

5. 합격률이 5%인 데이터에서 “전원 불합격” 모델의 정확도는?

5 Step 4. 전처리 파이프라인

목표 ⑤

문자열과 크기가 다른 숫자를, 모델이 계산할 수 있는 숫자 행렬로 바꾼다.

5.1 두 가지 변환

- **One-hot encoding**: "Marketing" → department_Marketing = 1. drop="first"는 기준 범주 열을 빼서 중복을 없앱니다 (응용 책의 dummy와 같음 [응용 0.8, 6.4]).
- **Standardization**: $z = (x - \bar{x})/SD$. 점수(29-100)와 경력(0-11)의 단위를 맞추니다 [흐름 3.1]. logistic의 규제(regularization)는 계수 크기에 벌점을 주므로 단위가 같아야 공평합니다.

```
pre = ColumnTransformer([
    ("cat", ____(drop="first"), cat),
    ("num", ____, num),
])
pre.fit(X_train)
print(pre.get_feature_names_out())
print(pre.transform(X_train.head(3)).round(2))
```

범주 → 0/1 열
숫자 → 평균 0, SD 1

실행 결과

```
['cat__gender_M' 'cat__department_Marketing' 'cat__degree_Master'
 'cat__degree_PhD' 'num__test_score' 'num__years_experience']
[[ 1.  0.  1.  0. -0.81 -0.78]
 [ 0.  1.  0.  0.  0.73  1.76]
 [ 1.  0.  0.  0. -0.04 -1.29]]
```

무슨 일이 일어났나

train 첫 행 (M, Engineering, Master, 56점, 3년)이 어떻게 바뀌었나:

원래 값	변환	결과
gender = M	one-hot	gender_M = 1
department = Engineering	기준 범주	department_Marketing = 0
degree = Master	one-hot	Master = 1, PhD = 0
test_score = 56	$(56 - 65.49)/11.66$	-0.81
years_experience = 3	$(3 - 4.53)/1.97$	-0.78

평균 65.49와 SD 11.66은 **train**에서만 계산한 값입니다 (pre.fit(X_train)). test에는 같은 숫자를 그대로 적용합니다.

※ test의 평균으로 표준화하면 test 정보가 학습에 새어 들어갑니다 (leakage). Pipeline에 넣으면 이 순서가 자동으로 지켜 집니다.

확인 질문

6. 표준화 없이 logistic regression을 돌리면 test_score 계수의 단위는 무엇인가?

6 Step 5. 학습과 예측

목표 ⑥

logistic regression을 학습시키고, test 지원자마다 합격 확률과 0/1 예측을 얻는다.

6.1 모델

$$P(\text{hired} = 1 \mid x) = \sigma(b_0 + b_1x_1 + \cdots + b_6x_6), \quad \sigma(z) = \frac{1}{1 + e^{-z}}.$$

`fit`은 train 436명의 likelihood를 최대화하는 b 를 찾습니다 (MLE, Newton류 반복) [응용 7.2]. 그다음 확률이 임계값(기본 0.5) 이상이면 1로 예측합니다.

```
logit = Pipeline([("pre", pre), ("clf", LogisticRegression(max_iter=1000))])
logit.__(X_train, y_train)
p = logit.__(X_test)[: , 1] # 합격 확률 (두 번째 열)
y_hat = (p >= 0.5).astype(int)
print(p[:5].round(3), y_hat[:5])
```

실행 결과

```
[0.502 0.286 0.535 0.091 0.427] [1 0 1 0 0]
```

무슨 일이 일어났나

1. `fit`: 파이프라인이 `pre.fit_transform(X_train) → clf.fit(행렬, y_train)` 순서로 실행.
2. `predict_proba(X_test)`: `pre.transform`(train의 평균/SD 사용) → 확률. 결과는 146×2 배열 (열 0 = 불합격 확률, 열 1 = 합격 확률). `[:, 1]`로 합격 쪽만.
3. test 첫 번째 지원자 (M, Engineering, Master, 73점, 3년, 실제 합격)의 계산:

$$z = \underbrace{-0.596}_{b_0} + \underbrace{0.297}_M + \underbrace{0.175}_{\text{Master}} + \underbrace{0.627 \times 0.644}_{\text{점수 } z=0.644} + \underbrace{0.351 \times (-0.779)}_{\text{경력 } z=-0.779} = 0.008,$$

$\sigma(0.008) = 0.502 \Rightarrow 0.5$ 이상이라 합격(1) 예측. 맞았습니다.

4. 다섯 번째 (M, Engineering, Master, 74점, 1년, 실제 합격): $0.427 \Rightarrow$ 불합격 예측. 틀렸습니다 (경력이 짧아서).

확인 질문

7. 위 계산에서 `department_Marketing` 항이 0인 이유는?

7 Step 6. 평가: 혼동행렬과 지표

목표 ⑦

146명에 대한 예측을 네 칸으로 나누고, 목적에 맞는 점수를 계산한다.

7.1 혼동행렬 (confusion matrix)

	예측 0 (불합격)	예측 1 (합격)
실제 0	TN = 95 (맞게 거절)	FP = 4 (잘못 합격)
실제 1	FN = 31 (놓침)	TP = 16 (맞게 합격)

※ sklearn의 출력 [[95 4] [31 16]]은 위 표와 같은 배치입니다: 행 = 실제, 열 = 예측.

7.2 지표

지표	식	질문	값
Accuracy	$\frac{TP+TN}{\text{전체}}$	전체 중 몇 % 맞혔나	$\frac{111}{146} = 0.760$
Precision	$\frac{TP}{TP+FP}$	합격이라 한 사람 중 진짜 합격자 비율	$\frac{16}{20} = 0.800$
Recall (TPR)	$\frac{TP}{TP+FN}$	진짜 합격자 중 찾아낸 비율	$\frac{16}{47} = 0.340$
F1	$\frac{2PR}{P+R}$	precision과 recall의 조화평균	0.478
AUC	ROC 곡선 아래 면적	합격자에게 더 높은 확률을 주는가	0.796

```
print(confusion_matrix(y_test, y_hat))
tn, fp, fn, tp = confusion_matrix(y_test, y_hat).ravel()
print("accuracy ", (___) / len(y_test)) # 직접 계산
print("precision", tp / (___))
print("recall   ", tp / (___))
print("AUC      ", round(roc_auc_score(y_test, ___), 3)) # 확률을 넣는다
```

실행 결과

```
[[95  4]
 [31 16]]
accuracy 0.76 precision 0.8 recall 0.34 AUC 0.796
```

무슨 일이 일어났나

- 정확도 0.760은 기준선 0.678보다 8%p 높습니다. 그런데 재현율 0.34: 실제 합격자 47명 중 31명을 놓쳤습니다.
- 모델이 “합격”이라고 말하는 데 매우 신중합니다 (146명 중 20명만). 그 20명 중 16명은 맞았으니 precision은 높습니다.
- AUC 0.796: 무작위로 합격자 한 명과 불합격자 한 명을 뽑았을 때, 합격자에게 더 높은 확률을 줄 확률이 약 80%. 임계값과 무관한 지표입니다.

※ 통계 연결: precision과 recall은 조건부확률입니다. $\text{recall} = P(\hat{y} = 1 \mid y = 1)$, $\text{precision} = P(y = 1 \mid \hat{y} = 1)$. 둘을 헷갈리는 것은 $P(A \mid B)$ 와 $P(B \mid A)$ 를 헷갈리는 것과 같습니다 (p-value 오해와 같은 구조 [흐름 11.5]).

확인 질문

- 채용 서류 1차 통과율이려면 precision과 recall 중 무엇을 더 중시해야 할까?

8 Step 7. 임계값과 ROC 곡선

목표 ⑧

0.5라는 기준선을 바꾸면 무엇이 바뀌는지 본다.

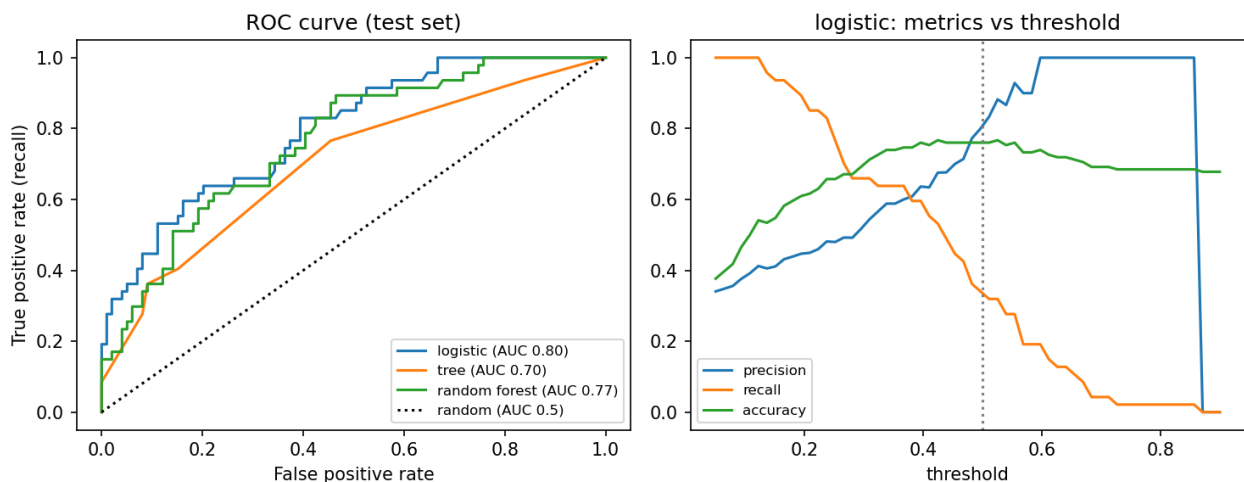
```
for thr in [0.3, 0.4, 0.5]:
    yh = (p >= __).astype(int)
    print(thr, confusion_matrix(y_test, yh).ravel(),
          "prec", round(precision_score(y_test, yh), 3), "rec", round(recall_score(y_test, yh),
          3))
```

실행 결과

```
0.3 [71 28 16 31] prec 0.525 rec 0.66
0.4 [83 16 19 28] prec 0.636 rec 0.596
0.5 [95  4 31 16] prec 0.8   rec 0.34
```

무슨 일이 일어났나

1. 확률 p 는 그대로이고, 자르는 선만 바꿉니다. 다시 학습하지 않습니다.
2. 임계값을 0.5 → 0.3으로 낮추면: 합격 예측이 20명 → 59명. 놓친 사람(FN) 31 → 16명으로 줄지만, 잘못 합격(FP) 4 → 28명으로 늘어납니다.
3. **precision-recall trade-off**: 하나를 올리면 다른 하나가 내려갑니다. 어느 쪽이 맞는지는 비용이 정합니다 (좋은 지원자를 놓치는 비용 vs 면접 비용).



- **ROC 곡선 (왼쪽)**: 임계값을 1에서 0으로 내리며 (FPR, TPR)을 찍은 것. 왼쪽 위로 붙을수록 좋은 모델, 대각선은 동전 던지기.
- **오른쪽**: 임계값에 따른 세 지표. 0.5 근처에서 정확도는 거의 평평하지만 recall은 급하게 떨어집니다.

※ 통계 연결: 임계값 선택은 가설검정의 α 선택과 같은 구조입니다. FP = Type I error, FN = Type II error, recall = power [흐름 11.4].

확인 질문

9. 임계값을 0으로 하면 confusion matrix는 어떻게 되나? recall과 precision은?

9 Step 8. 다른 모델과 교차검증

목표 ⑨

결정트리, 랜덤포레스트와 비교하되, test 한 번이 아니라 **5번** 나눠서 공정하게 비교한다.

9.1 교차검증 (cross-validation)

582행을 5조각으로 나누고, 한 조각씩 돌아가며 평가용으로 씁니다. 5개 점수의 평균과 흩어짐을 봅니다.

	조각 1	조각 2	조각 3	조각 4	조각 5
1회	평가	학습	학습	학습	학습
2회	학습	평가	학습	학습	학습
⋮			⋮		
5회	학습	학습	학습	학습	평가

```
pre_tree = ColumnTransformer([("cat", OneHotEncoder(drop="first"), cat)],
                              remainder="passthrough")
tree = Pipeline([("pre", pre_tree), ("clf", DecisionTreeClassifier(max_depth=___,
min_samples_leaf=20, random_state=0))])
forest = Pipeline([("pre", pre_tree), ("clf", RandomForestClassifier(n_estimators=300,
min_samples_leaf=10, random_state=0))])
cv = StratifiedKFold(___, shuffle=True, random_state=0)
for name, m in [("logit", logit), ("tree", tree), ("forest", forest)]:
    s = cross_val_score(m, ___, ___, cv=cv, scoring="roc_auc")
    print(name, s.round(3), "mean", s.mean().round(3))
```

전체 X, y

실행 결과

```
logit [0.781 0.816 0.659 0.783 0.749] mean 0.758
tree [0.715 0.773 0.656 0.738 0.685] mean 0.714
forest [0.783 0.81 0.671 0.78 0.769] mean 0.763
```

무슨 일이 일어났나

1. 모델마다 5번 학습·평가 = 총 15번 학습. 매번 파이프라인 전체(전처리 포함)를 그 회차의 학습 조각으로만 fit합니다 (leakage 없음).
2. 같은 cv 객체를 쓰므로 세 모델이 같은 조각으로 비교됩니다.
3. 조각 3에서 세 모델 모두 낮음 (0.66 근처): 그 조각이 어려운 것이지 모델 탓이 아닙니다. 점수가 0.66–0.82로 흔들린다는 것 자체가 중요한 정보입니다.
4. logistic 0.758 vs forest 0.763: 차이(0.005)가 회차 간 흔들림(SD 약 0.05)보다 훨씬 작습니다. 사실상 같은 성능이고, 해석이 쉬운 logistic을 고릅니다.

※ 트리 모형은 표준화가 필요 없어서 (remainder="passthrough") 숫자 열을 그대로 넘깁니다. 트리는 값의 순서만 보고 분할하기 때문입니다 [알고 6, 7].

※ 통계 연결: “두 모델의 차이가 우연 범위인가”는 가설검정의 질문입니다. 5개 점수의 차이에 paired *t*-test를 할 수도 있습니다 [흐름 12 ⑥].

확인 질문

10. test set 점수(logit 0.796)와 교차검증 평균(0.758)이 다른 이유는?

10 Step 9. 해석: 계수와 중요도

목표 ⑩

모델이 무엇을 보고 예측하는지 확인한다.

```
names = logit.named_steps["pre"].get_feature_names_out()
coef = logit.named_steps["clf"].coef_[0]
print(pd.DataFrame({"feature": names, "coef": coef.round(3),
                    "odds_ratio": np.__(coef).round(2)}))

pi = permutation_importance(logit, X_test, y_test, scoring="roc_auc",
                             n_repeats=30, random_state=0)
imp = pd.Series(pi.importances_mean, index=X.columns)
print(imp.round(3).sort_values(ascending=False))
```

실행 결과

	feature	coef	odds_ratio
0	cat__gender_M	0.297	1.35
1	cat__department_Marketing	-1.256	0.28
2	cat__degree_Master	0.175	1.19
3	cat__degree_PhD	0.298	1.35
4	num__test_score	0.627	1.87
5	num__years_experience	0.351	1.42
department	0.106	test_score	0.095
years_experience	0.032	degree	0.016
gender	0.010		

무슨 일이 일어났나

- Odds ratio:** Marketing이면 합격 odds가 0.28배, 점수가 1 SD (약 11.7점) 높으면 1.87배, 남성이면 1.35배 [응용 7.3].
- Permutation importance:** 한 열의 값을 test 안에서 무작위로 섞었을 때 AUC가 얼마나 떨어지나. 부서를 섞으면 0.106 하락, 성별은 0.010.
- 계수는 “한 단위 변할 때의 효과”, permutation은 “그 정보가 없으면 예측이 얼마나 나빠지나”. 성별 계수가 0이 아닌데 중요도가 작은 이유: 성별 정보의 상당 부분이 부서에 이미 들어 있기 때문입니다 (성별 $\times$ 부서 $\chi^2 = 132$ [흐름 12 ⑧]).

※ 응용 책의 statsmodels logit은 남성 odds ratio 1.60이었습니다. 여기서 1.35인 이유: (1) train 436명만 썼고, (2) sklearn의 LogisticRegression은 기본으로 L2 규제(C=1)를 걸어 계수를 0 쪽으로 줄입니다. 규제 = Bayesian prior [응용 8.6]. 해석과 검정이 목적이면 statsmodels, 예측이 목적이면 sklearn을 씁니다.

확인 질문

- 섞기(permutation)를 train이 아니라 test에서 하는 이유는?

11 Step 10. 공정성 감사 (fairness audit)

목표 ①

모델이 성별에 따라 다르게 틀리는지 확인한다. 이 워크북의 핵심 단계.

11.1 그룹별로 혼동행렬 쪼개기

```
def group_report(p, thr=0.5):
    t = X_test.copy()
    t["y"] = y_test.values
    t["y_hat"] = (p >= thr).astype(int)
    return t.groupby(__).apply(lambda g: pd.Series({
        "n": len(g),
        "actual_rate": g["y"].mean(),
        "selection_rate": g["y_hat"].mean(),
        "TPR": g.loc[g["y"] == __, "y_hat"].mean(),
        "FPR": g.loc[g["y"] == __, "y_hat"].mean(),
    }), include_groups=False).round(3)
print(group_report(p))
```

실제 합격자 중 합격 예측
실제 불합격자 중 합격 예측

실행 결과

	n	actual_rate	selection_rate	TPR	FPR
F	77.0	0.182	0.026	0.071	0.016
M	69.0	0.478	0.261	0.455	0.083

무슨 일이 일어났나

1. test의 여성 77명 중 실제 합격자 14명 (18.2%), 남성 69명 중 33명 (47.8%).
2. 모델이 합격이라고 예측한 사람: 여성 2명 (2.6%), 남성 18명 (26.1%).
3. **TPR (재현율)**: 실제 합격한 여성 14명 중 모델이 찾아낸 사람은 1명 (0.071). 실제 합격한 남성 33명 중에서는 15명 (0.455).
4. 즉 똑같이 합격할 사람이라도, 여성이면 모델에게 거의 발견되지 않습니다. 이것이 **equal opportunity** (그룹 간 TPR이 같아야 한다) 위반입니다 [응용 9.1].

11.2 성별을 빼면 해결될까? (fairness through unawareness)

```
pre2 = ColumnTransformer([("cat", OneHotEncoder(drop="first"), [__]),
    ("num", StandardScaler(), num)])
logit2 = Pipeline([("pre", pre2), ("clf", LogisticRegression(max_iter=1000))])
logit2.fit(X_train, y_train)
p2 = logit2.predict_proba(X_test)[: , 1]
print(group_report(p2))
```

실행 결과

	n	actual_rate	selection_rate	TPR	FPR
F	77.0	0.182	0.091	0.071	0.095
M	69.0	0.478	0.246	0.424	0.083

무슨 일이 일어났나

1. 성별 열을 빼도 여성 TPR은 그대로 **0.071**, 남성은 0.424. 격차가 거의 안 줄었습니다.
2. 이유: **proxy (대리) 변수**. 여성의 68%가 Marketing에 지원했고 Marketing은 합격률이 낮습니다. 모델은 “Marketing”을 통해 성별 정보를 간접적으로 씁니다.
3. 오히려 여성 FPR이 0.016 → 0.095로 올라, 여성에게서 “잘못 합격”이 늘었습니다. 성별을 빼는 것만으로는 공정해지지 않습니다.

11.3 반사실 테스트: 성별만 바꾸면?

```
X_flip = X_test.copy()
X_flip["gender"] = X_flip["gender"].map({"M": ___, "F": ___})
p_flip = logit.predict_proba(X_flip)[ :, 1]
print("결정이 바뀐 사람:", ((p_flip >= 0.5) != (p >= 0.5)).sum(), "/", len(X_test))
```

실행 결과

결정이 바뀐 사람: 14 / 146

※ 다른 조건은 모두 같고 성별만 바꿨는데 146명 중 14명(약 10%)의 합불이 뒤집혔습니다. 평균적으로 여성 → 남성으로 바꾸면 합격 확률 +4.6%p, 남성 → 여성이면 -5.9%p. 응용 책 `assign(gender="M")` 계산과 같은 반사실 논리입니다 [응용 7장 pandas 코드].

확인 질문

12. 여성 TPR 0.071은 실제 합격 여성 14명에 기반한다. 이 숫자를 발표할 때 함께 말해야 할 주의점은?

12 정리와 발표

목표 ⑫

분류 결과를 “bias in hiring” 질문에 대한 답으로 바꾼다.

12.1 이번 실습으로 알게 된 것

단계	결과
기준선	전원 불합격 예측으로도 정확도 0.678 → 정확도만으로는 판단 불가
logistic	정확도 0.760, AUC 0.796 (CV 0.758). 합격자를 34%만 찾음
모델 비교	tree 0.714, forest 0.763: logistic과 사실상 같음 → 해석 쉬운 logistic 선택
중요 특성	부서 > 점수 > 경력 > 학위 > 성별 (permutation)
공정성	실제 합격 여성 중 1/14만 발견 (TPR 0.07 vs 남성 0.46)
성별 제거	격차 유지 (부서가 proxy)
반사실	성별만 바꾸면 10%의 결정이 뒤집힘

12.2 발표용 문장 (예시)

이렇게 말하면 된다

“과거 채용 데이터로 합격 예측 모델을 학습시키면 정확도 76%, AUC 0.80의 모델이 나온다. 그러나 이 모델은 실제로 합격한 여성 14명 중 1명만 합격으로 예측했고, 남성은 33명 중 15명을 찾았다. 성별 변수를 제거해도 지원 부서가 성별의 대리 변수로 작동해 격차가 유지된다. 다른 조건은 같고 성별만 바뀌어도 약 10%의 결정이 뒤집힌다. 즉, 이 데이터로 자동화된 채용 시스템을 만들면 과거의 성별 격차를 그대로 재생산할 위험이 크다. 단, test의 여성 합격자가 14명뿐이라 수치의 불확실성이 크므로, 통계 분석(응용 책 6-8장)의 결론과 함께 제시한다.”

12.3 다음에 해 볼 것

- 임계값을 그룹별로 달리해서 TPR을 맞추기 (post-processing). 공정성은 좋아지지만 “성별에 따라 다른 기준”이라는 법적·윤리적 문제가 생깁니다 [응용 9.1 impossibility].
- 부서별로 따로 모델 만들기: Engineering 안에서의 격차만 보기 (응용 책 interaction과 비교).
- 공정성 지표에 bootstrap 신뢰구간 붙이기 [흐름 9.2]: 14명짜리 TPR이 얼마나 흔들리는지.

13 확인 질문 답과 빈칸 정답

13.1 확인 질문

- 회귀 (정답이 숫자).
- 아닙니다. 정확도는 과거 결정을 얼마나 잘 따라 하느냐이고, 과거 결정이 편향이면 정확한 모델도 편향입니다 (11장).
- 네. `drop`은 지정한 열을 빼고 나머지를 남깁니다. 열이 많을 때 편합니다.
- 우연히 `test`에 합격자가 너무 적거나 많게 들어갈 수 있습니다. 합격자 5%(582명 중 약 29명)면 `test` 146명에 합격자가 3명만 들어갈 수도 있어서, `recall` 같은 지표가 몇 명에 좌우됩니다.
- 95%. 그래서 불균형 데이터에서는 정확도 대신 `recall`, `precision`, `AUC`를 봅니다.
- “점수 1점당 `log-odds` 변화”. 표준화하면 “1 SD(약 11.7점)당”이 됩니다. 예측 확률은 같지만 (규제가 없다면) 계수의 숫자만 바뀝니다.
- 이 지원자는 Engineering (기준 범주)이라 `department_Marketing = 0`이고, $-1.256 \times 0 = 0$ 입니다.
- 1차 통과는 “좋은 사람을 놓치지 않기”가 중요하므로 **`recall`**. 최종 합격 결정이라면 `precision`이 더 중요할 수 있습니다.
- 모두 1로 예측하므로 $TN = FN = 0$, `[0 99 0 47]`. `recall = 1`, `precision = 47/146 = 0.322` (합격률).
- `test` 146명은 한 번의 무작위 분할 결과라 운이 섞입니다. CV는 5번의 평균이라 더 안정적인 추정입니다. 이번 `test` 분할은 쉬운 편이었습니다.
- 모델이 본 적 없는 데이터에서 “그 정보가 예측에 얼마나 필요한가”를 재야 하기 때문입니다. `train`에서 재면 외운 패턴까지 중요하다고 나옵니다.
- 14명 중 1명이라 한두 명만 달라져도 `TPR`이 크게 바뀝니다 ($1/14 = 0.07$, $3/14 = 0.21$). 표본 크기를 함께 말하고, 가능하면 교차검증이나 `bootstrap`으로 구간을 제시합니다.

13.2 빈칸 정답

Step	빈칸
1	<code>dropna, cat + num, "hired"</code>
2	<code>0.25, y</code>
3	<code>"most_frequent"</code>
4	<code>OneHotEncoder, StandardScaler</code>
5	<code>fit, predict_proba, 0.5</code>
6	<code>tp + tn, tp + fp, tp + fn, p</code>
7	<code>thr</code>
8	<code>3, 5, X, y</code>
9	<code>exp</code>
10	<code>"gender", 1, 0, "department", "degree", "F", "M"</code>

※ 전체 정답 코드와 출력은 `classification_solution.ipynb`에 있습니다.